

82. Sinus, cosinus, une seule table

Pas de virgule flottante, pas de type négatif : une table gravée y suffit.

1. Ce qu'il faut comprendre

Il n'y a ni `sin()` ni `cos()` ici, et pas de type signé non plus — impossible d'écrire un nombre négatif directement. La solution tient dans **une table gravée** : chaque case vaut `40 + 40 * sin(angle)`, toujours entre 0 et 80, jamais en dessous de zéro.

`cos(angle) = sin(angle + 90°)` — et 90°, c'est un quart de tour. Avec trente-deux pas pour un tour complet, un quart de tour fait huit pas. Pas besoin d'une deuxième table : `cosinus` relit la MÊME, huit cases plus loin, avec `%` pour revenir au début quand ça déborde.

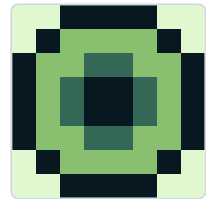
Le reste est déjà connu : `aAvant` pour agir **au front** — un seul basculement par appui, et non un par image —, et deux textes de même longueur, pour que rien ne traîne de l'un à l'autre.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1558 octets** de cartouche.

1. UN DESSIN — BALLE

```
Tuile BALLE = { ".#####.", ".#---#.", "#--++--#", "#-+++-#", "#-+++-#", "#-+++-#", ".#---#.", ".#####." };
```



à l'écran

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — SINUS

```
const uint8_t SINUS[] = {
    40, 48, 55, 62, 68, 73, 77, 79,
    80, 79, 77, 73, 68, 62, 55, 48,
    40, 32, 25, 18, 12, 7, 3, 1,
    0, 1, 3, 7, 12, 18, 25, 32,
};
```

3. UNE VARIABLE — X

```
uint8_t x = 0;
```

4. UNE VARIABLE — ANGLE

```
uint8_t angle = 0;
```

5. UNE VARIABLE — COSINUS

```
uint8_t cosinus = 0;
```

6. UNE VARIABLE — AAVANT

```
uint8_t aAvant = 0;
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        uint8_t a = bouton(A);
        if (a && !aAvant) cosinus = !cosinus;
        aAvant = a;

        if (cosinus) texte(1, 1, "A : COSINUS");
        else texte(1, 1, "A : SINUS  ");

        uint8_t indice = cosinus ? (angle + 8) % 32 : angle;
        uint8_t y = 28 + SINUS[indice];

        sprite(0, x, y, BALLE);

        x++;
        if (x > 152) x = 0;

        angle++;
        if (angle >= sizeof(SINUS)) angle = 0;
    }

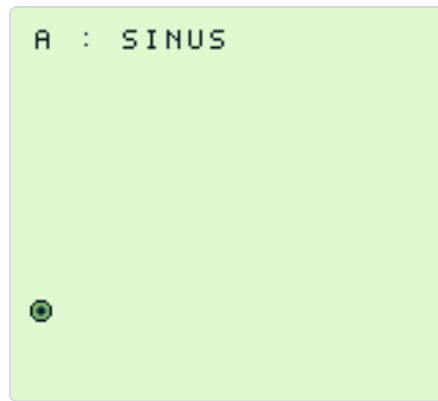
    return 0;
}
```

3. Ce qui se passe, étape par étape

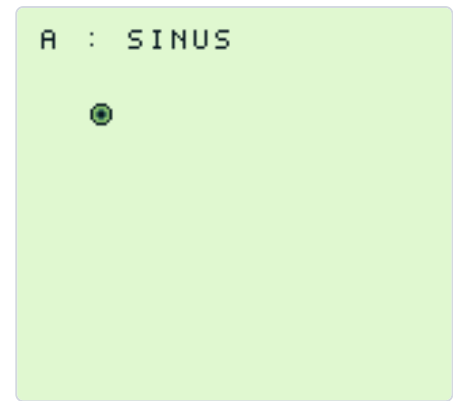
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



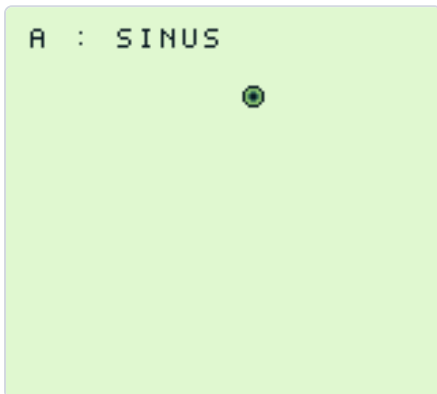
la toute première image — la console vient de démarrer — image 1



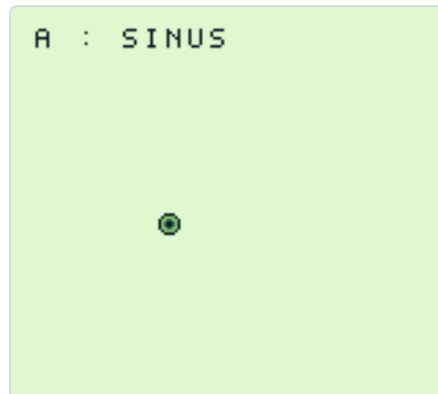
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



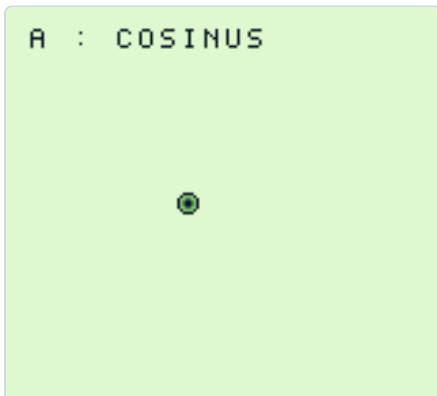
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Une balle qui ondule en traversant l'écran ; A bascule « SINUS » en « COSINUS », et elle saute d'un quart de tour.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le titre de départ est SINUS

✓ elle porte la tuile dessinée

✓ elle reste dans la plage de la table (min 28, max 108)

✓ elle ondule vraiment, et ne file pas tout droit

✓ A bascule sur COSINUS, au front

✓ le titre change, à la même longueur